

IIIT Kottayam — BTP Discussion 2

iRDRC: Markov Decision Process

Deep Dive

Joel James & Majida Bee

29 August 2026

*Focus: Complete mathematical formulation of the MDP, why it works,
algorithmic aspects*

Recap: The iRDRC Problem

- ▶ **Problem:** AV needs radar + communication on shared resource
- ▶ **Environment:** Weather, road, speed, moving objects (binary states)
- ▶ **Objective:** Maximize throughput while minimizing miss-detection risk
- ▶ **Approach:** Adaptive learned policy via Deep Q-Network (DQN)
- ▶ **Key Insight:** Fixed scheduling fails under dynamic uncertainty

Meeting 1: System model, environment model (Eq. 1), MDP intro

Today: Deep understanding of MDP, why each component matters,
DQN algorithm

Why Markov Decision Process (MDP)?

The Markov Property: The future depends only on the current state, not on history.

Why this matters for iRDRC:

1. Queue state d captures current backlog; only d matters, not arrival history
2. Channel state c is independent each step; history doesn't predict next c
3. Environmental factors (r, w, v, m) are resampled; each step is a fresh draw
4. Radar detection is perfect; if on and event exists, we catch it

Result: Given current state s and action a , the next state s' is probabilistically determined. This allows us to use RL algorithms that exploit the Markov property.

The MDP Tuple: $\langle S, A, P, R, \gamma \rangle$

A Markov Decision Process is defined by five components that describe a complete sequential decision problem.

Component	Symbol	Meaning (iRDRC)	Example
State Space	S	All possible observations	$(d = 5, c = 0, r = 0, w = 1, v = 1, m = 0)$
Action Space	A	All possible choices	$\{0 : \text{comm}, 1 : \text{radar}\}$
Transition	$P(s' s, a)$	How state evolves	$P(d = 4 d = 5, a = 0, c = 0)$
Reward	$R(s, a, s')$	Immediate reward	+2 (good comm), -50 (missed event)
Discount	γ	Weight on future rewards	$\gamma \approx 0.99$

State Space (Equation 2)

Definition:

$$S = \{(d, c, r, w, v, m) : d \in \{0, \dots, 10\}, c, r, w, v, m \in \{0, 1\}\}$$

State Space Size Calculation:

- ▶ d (queue): 11 possible values $(0, 1, 2, \dots, 10)$
- ▶ c (channel): 2 possible values $(0 = \text{good}, 1 = \text{bad})$
- ▶ r (road): 2 possible values
- ▶ w (weather): 2 possible values
- ▶ v (speed): 2 possible values
- ▶ m (moving objects): 2 possible values

Total: $11 \times 2^5 = 352$ **states**

352 states is manageable but large enough to benefit from function approximation (neural networks) instead of tabular methods.

Action Space (Simple but Powerful)

Definition:

$$A = \{0, 1\}$$

Action	Mode	What Happens
$a = 0$	Communication	Transmit queued packets; channel determines count
$a = 1$	Radar	Scan for obstacles; detect if event exists

Performance Impact:

- Communication: Increases throughput, increases miss-detection risk
- Radar: Decreases throughput, decreases miss-detection risk

Central Design Decision: Each time step, the AV chooses exactly one mode. This binary action is the lever the learning algorithm will adjust.

Reward Function (Equation 3)

The Learning Signal: Translates engineering objectives (throughput + safety) into numbers a learning algorithm can optimize.

Action	Situation	Reward	Interpretation
Comm	Good channel, no event	$+r_1 = +2$	Success: 4 packets sent
Comm	Bad channel, no event	$+r_2 = +1$	Partial: 2 packets sent
Comm	Event occurs	$-r_3 = -50$	Catastrophic: missed obstacle
Radar	No event	0	Radar cost: found nothing
Radar	Event occurs (b unfav.)	$+r_4(b + 1)$	Success: scales with risk

Key Design Choices:

- ▶ $-50 \gg r_1, r_2$ (Safety-First): Missing event is 25–50 $\times$ worse than missing packets
- ▶ Reward scales with risk: Detected event in high-risk ($b = 4$) earns +25, in low-risk ($b = 0$) earns +5

Objective: Discounted Return (Equation 4)

Goal: Find policy π that maximizes expected cumulative reward over many time steps.

Discounted Return (Value of Policy π):

$$G(\pi) = E \left\{ \sum_{t=0}^T \gamma^t r_{t+1}(\pi) \right\}$$

Components:

- ▶ $r_{t+1}(\pi)$: Immediate reward at time $t + 1$ under policy π
- ▶ $\gamma \in (0, 1)$: Discount factor (paper doesn't specify; typical ≈ 0.99)
- ▶ γ^t : Exponential decay; rewards 10 steps away worth $\gamma^{10} \approx 0.90 \times$ immediate
- ▶ T : Time horizon (episode length; likely 500–1000 steps)

Optimal Policy:

$$\pi^* = \arg \max_{\pi} G(\pi)$$

Why Discount the Future? (γ Explained)

What γ Controls:

- ▶ $\gamma \rightarrow 1.0$ (e.g., **0.99**): Future rewards nearly as valuable as immediate rewards. Policy is far-sighted. Over $T = 1000$, reward at step 999 is still worth $\gamma^{999} \approx 0.0005$ of immediate reward.
- ▶ $\gamma \rightarrow 0.0$ (e.g., **0.5**): Only immediate rewards matter. Policy is myopic; will choose any immediate gain even if it ruins future prospects.

For iRDRC:

- ▶ $\gamma \approx 0.99$ makes sense: a crash today affects all future steps
- ▶ If $\gamma = 0.5$, policy might communicate recklessly to gain +2, knowing -50 crash is only 50% as important
- ▶ $\gamma \approx 0.99$ ensures policy balances immediate throughput against long-term safety

From MDP to Policy: Q-Values and π

Action-Value Function $Q(s, a)$: Expected discounted return from state s , action a , following optimal policy.

$$Q^*(s, a) = E \left\{ r + \gamma \max_{a'} Q^*(s', a') \right\}$$

Intuition:

- ▶ $Q(d = 0, a = 0)$ high: queue empty, communication valuable
- ▶ $Q(d = 10, a = 0)$ lower: queue full
- ▶ $Q(w = 1, v = 1, a = 0)$ very low: unfavorable weather/speed, communication
- ▶ $Q(w = 1, v = 1, a = 1)$ higher: radar protects from risk

Optimal Policy π^* : $\pi^*(s) = \arg \max_{a \in A} Q^*(s, a)$

Pick action with max Q-value. **Challenge:** Learn Q^* from experience.

Environment Dynamics: State Transitions $P(s'|s, a)$

Queue Dynamics (Deterministic):

$$d'_{t+1} = \min(d_t - \text{transmission} + \text{arrivals}, D)$$

Transmission: $\nu_1 = 4$ if $c = 0$ (good), $\nu_2 = 2$ if $c = 1$ (bad). Arrivals $\sim \text{Poisson}(\lambda_d = 1)$.

Channel Dynamics (Stochastic, Independent):

$$c'_{t+1} \sim \text{Bernoulli}(p_c = 0.1)$$

$P(c' = 1|c, a) = 0.1$ (independent of c and a).

Environmental Factors (Stochastic, Independent): Each of r, w, v, m is resampled: $P(x' = 1) = 1 - \tau_x$. All factors independent (Markov property).

Unexpected Event Generation (Stochastic):

$$\oplus_t \sim \text{Bernoulli}(P(\oplus))$$

If $a_t = 1$ (radar) and $\oplus_t = 1$, detect. If $a_t = 0$ (communication) and $\oplus_t = 1$, miss (−50 penalty).

Tabular Q-Learning vs. Deep Q-Network (DQN)

Aspect	Tabular Q-Learning	Deep Q-Network
Storage	352 table entries	Neural network weights
Update	$Q \leftarrow Q + \alpha[r + \gamma \max_{a'} Q(s', a') - Q]$	Same rule, network output
Converge	~280 episodes	~170 episodes
Scalability	352 states OK; larger impractical	Scales to large state spaces
Data Efficiency	One update per experience	Mini-batch updates (variance reduction)

Bottom Line: For iRDRC with 352 states, both work. DQN is faster. Both exploit MDP structure; learning from experience is enough.

DQN Network Update (Equation 5)

Update Rule:

$$\theta_{t+1} = \theta_t + \alpha [y_t - Q(s_t, a_t; \theta_t)] \nabla Q(s_t, a_t, \theta_t)$$

Where:

- ▶ θ_t : Network weights at time t
- ▶ α : Learning rate (e.g., 0.001)
- ▶ $y_t = r_t + \gamma \max_{a'} Q(s_{t+1}, a'; \theta_t^-)$: Target value
- ▶ θ_t^- : **Target network** weights (copied from online network periodically, e.g., every 10k steps)
- ▶ $\nabla Q(\dots)$: Gradient of Q-value w.r.t. weights

Key Innovation: Target Network

Why two networks? Target network provides stable target for learning. Without it, y_t changes as θ changes, causing oscillations. By updating θ^- only periodically, we stabilize training.

DQN Training Loop: 7 Steps per Experience

Step	Action	Details
1	Observe state	AV measures (d, c, r, w, v, m)
2	Estimate Q	Feed state to network $\rightarrow$ get $Q(s, 0)$ and $Q(s, 1)$
3	Choose action	ϵ -greedy: explore with prob ϵ , exploit ($\max Q$) with prob $1 - \epsilon$
4	Interact	Execute a ; environment returns reward r and next state s'
5	Store memory	Add (s, a, r, s') to replay buffer (circular, $\sim 10k$)
6	Sample/train	Mini-batch (~ 32) from buffer; compute loss and update θ
7	Repeat	Continue episodes until cumulative reward converges

Why Replay Memory? Random sampling decorrelates data, reducing variance and stabilizing learning.

Summary: Complete MDP for iRDRC

Component	Definition	Size/Form	Implementation Note
State Space	(d, c, r, w, v, m)	352 discrete states	Fully observable; measured sensors
Action Space	$\{0 : \text{comm}, 1 : \text{radar}\}$	2 actions	Binary choice each step
Reward	$r_1 = 2, r_2 = 1$ $r_3 = -50, r_4 = 5(b + 1)$	-50 to +25	Safety-first asymmetry
Transition	Queue (det.); channel, env (stoch.)	Eq. 1 for event prob.	Markov: future indep. of history
Discount	Typical ≈ 0.99	Scalar in $[0, 1)$	Not specified; use typical value
Solver	DQN (Keras/PyTorch)	Neural net $(6 \rightarrow 2)$	170 episodes to converge

Key Takeaways: Why MDP + DQN Works

1. **MDP Captures Structure:** Markov property holds; past doesn't matter, only current state. Algorithms can plan without knowing transition probabilities.
2. **Reward Design Encodes Goals:** Asymmetric reward ($-50 \gg +1, +2$) implements safety priority. Learning discovers the right balance.
3. **Discount Factor Ensures Long-Horizon Planning:** $\gamma \approx 0.99$ makes policy care about future crashes. Without it, policy would be greedy and reckless.
4. **DQN Scales Beyond Tabular Methods:** 352 states manageable for both Q-learning and DQN. DQN faster (170 vs. 280 episodes) due to mini-batch updates and replay memory stabilization.
5. **Adaptation Emerges from Learning:** Policy not programmed to “use radar when risky.” It learns this strategy by optimizing cumulative reward, adapting to any condition

Modeling Gaps to Address Before Implementation

Gap	What's Missing	Action for Implementation
Probability	$p_r^0, p_r^1, p_m^0, p_m^1$ not specified	Estimate: $p^0 = 0.01, p^1 = 0.05$
State Evolution	τ_i (prob. favorable) not given	Assume $\tau_r \approx 0.7, \tau_w \approx 0.8, \tau_v \approx 0.6, \tau_m \approx 0.7$
Discount γ	Not specified in paper	Use $\gamma \approx 0.99$ (standard)
Episode Length T	Not explicit; 500–1000 steps	Experiment with $T = 500$, measure convergence
DQN Params	Learning rate, batch, buffer, ϵ -decay unspecified	Defaults: $\alpha = 0.001, \text{batch} = 32, \text{buffer} = 10k$
Network Arch	Hidden layers/units not	2 layers, 64 units each ($6 \rightarrow 64 \rightarrow 64 \rightarrow 2$)

Thank You

Questions & Discussion